

Head Soccer Project



Joaquín Avilés de la Fuente
Bruno Krawietz

28 giugno 2026

Contents

1	Introduction & Environment Description	5
1.1	Context and Motivation	5
1.2	Technical Environment Description	5
1.3	The Main Loop & Frame-rate Independence	5
1.3.1	Synchronization with requestAnimationFrame	6
1.3.2	Mathematical Formulation of Delta Time	6
1.3.3	State Update Distribution and Code Implementation	6
2	External Libraries & Assets	7
2.1	Third-Party Dependencies	7
2.1.1	gl-matrix.js	7
2.2	Asset Strategy: 100% Procedural Generation	7
2.2.1	Geometry Generation	7
2.2.2	Texture Creation and Virtual Canvas Routing	8
3	Geometric Modeling & Primitive Reuse	9
3.1	Abstraction of Base Shapes via Code	9
3.1.1	Boxes and Planes	9
3.1.2	Spheres and Curved Shapes via Trigonometric Latitudes	9
3.2	The Reuse Approach: Instancing via Transformations	10
3.2.1	Single Topology in Memory	10
3.2.2	Deformation via Matrices	10
4	Affine Transformations & Hierarchical Scene Graph	11
4.1	The Model Matrix: Encapsulation in SceneObject	11
4.2	Hierarchical Structure of the Players	11
4.3	Movement Propagation: Matrix Cloning over Traditional Stacks	11
5	Procedural Trigonometric Animations	13
5.1	Movement without an External Skeleton	13
5.2	The Walk Cycle: Sine Wave Alternation	13
5.3	The Head Bobbing Effect: Absolute Values for Vertical Impact	13
6	Advanced Lighting & Shader Pipeline	15
6.1	The Shader Pipeline: Data Flow and Interpolation	15
6.2	Conceptual Breakdown of the Blinn-Phong Model	15
6.3	Multi-Light System and Distance Attenuation Code	16
7	Texturing Techniques & Camera Systems	17
7.1	Real-Time Texture Generation via Canvas API	17
7.2	Camera Setup and Look-At Space	17
7.3	The Dynamic Stadium Camera Logic	17
8	Physics Engine & Collision Responses	19
8.1	Kinematics and Euler Integration	19
8.2	Geometric Collision Detection	19
8.3	Momentum Transfer and Kick Modifications	19

8.4	Lock Resolution: The Sandwich Effect	20
9	User Manual & Deployment Guide	21
9.1	System Requirements	21
9.2	Installation and Deployment Instructions	21
9.3	Interactive Game Controls Guide	21
9.3.1	Environmental Modification Triggers	21

1. Introduction & Environment Description

1.1 Context and Motivation

The project **3D Head Soccer** is an interactive, real-time sports simulation game inspired by classical arcade mechanics. It features two stylized, big-headed characters competing in a confined stadium with the primary objective of scoring a ball into the opponent's goalpost. From an academic perspective, the motivation behind this project is to bridge the gap between abstract computer graphics theory and practical software engineering.

Rather than treating the graphics pipeline as a black box, this project implements a complete, self-contained rendering engine. The core objective is to master the mathematical foundations of interactive graphics—specifically coordinate spaces, affine transformations, hierarchical modeling, procedural lighting, and real-time collision response—by implementing them manually at the lowest programmatic level available in modern web environments.

1.2 Technical Environment Description

To guarantee a comprehensive understanding of the graphics pipeline, the software environment rejects high-level commercial frameworks or engines such as Three.js, Babylon.js, or Unity. Instead, the game is built entirely upon **basic, native WebGL** and vanilla **JavaScript (ES6)**.

WebGL acts as a low-level rasterization API that maps directly to the hardware acceleration capabilities of the Graphics Processing Unit (GPU). This architectural choice demands that the development team manually orchestrate operations that high-level engines typically automate, including:

- Defining raw vertex streams, normals, and texture coordinates.
- Designing, compiling, and linking custom programmable Shaders written in **GLSL** (OpenGL Shading Language).
- Directing data communication between CPU memory and GPU registers via **uniform** and **attribute** variables.

The execution context relies on the HTML5 `<canvas>` element embedded within the `index.html` layout, which acts as the target rendering surface. Performance is maximized by enforcing hardware-accelerated processing without external plug-ins, ensuring cross-platform capability across any modern, standards-compliant web browser.

1.3 The Main Loop & Frame-rate Independence

A critical requirement of interactive graphical systems is the maintenance of a stable, deterministic simulation loop. In our application, this is achieved by orchestrating an asynchronous execution cycle anchored to the browser's native `requestAnimationFrame` API, located within the core initialization routine in `main.js`.

1.3.1 Synchronization with requestAnimationFrame

Unlike traditional interval timers (`setInterval`), which execute arbitrarily and cause screen tearing or stuttering, `requestAnimationFrame` synchronizes the game's updates directly with the refresh rate of the monitor (typically 60Hz, 120Hz, or higher). This minimizes latent rendering overhead and ensures that drawing calls only occur when the display is ready for a repaint.

1.3.2 Mathematical Formulation of Delta Time

To prevent the game logic, physics updates, and animations from fluctuating based on the hardware's execution speed (Frame-rate Dependency), a strict **Frame-rate Independence** strategy is implemented. This is achieved by computing a temporal scale factor known as *Delta Time* (Δt).

At each iteration of the main loop, the application samples the high-resolution system clock using the absolute timestamp provided by the API:

$$\Delta t = \frac{T_{\text{current}} - T_{\text{previous}}}{1000} \quad (1.1)$$

Where T_{current} and T_{previous} represent the time execution values measured in milliseconds. Dividing by 1000 normalizes Δt into seconds.

1.3.3 State Update Distribution and Code Implementation

Within `main.js`, this calculated Δt is distributed to the physical and analytical subsystems. Any kinematic update—such as updating a particle's position based on its velocity vector $\mathbf{v}$ —is scaled linearly by Δt :

$$\mathbf{p}_{\text{new}} = \mathbf{p}_{\text{old}} + \mathbf{v} \cdot \Delta t \quad (1.2)$$

Consequently, if the frame rate drops under heavy rendering loads, Δt increases proportionally, ensuring that objects traverse the exact same distance in physical world space per absolute unit of time.

The following JavaScript snippet from our `main.js` file shows exactly how our core routine handles this temporal synchronization, updates the logical state, adjusts the camera viewport, and clears the buffers before sending the draw commands to the GPU:

```

1 function render(currentTime) {
2   if (!gameState || !camera) return;
3
4   // Calculate deltaTime and cap its step for physics stability
5   let deltaTime = lastTime ? (currentTime - lastTime) / 1000.0 : 0.016;
6   deltaTime = Math.min(deltaTime, 0.032);
7   lastTime = currentTime;
8
9   // 1. Process game logic, input actions, and custom physics
10  gameState.update(currentTime);
11
12  // 2. Adjust camera matrices based on the ball's position
13  camera.update(gameState);
14
15  // 3. Clear WebGL color and depth buffers for the new frame
16  gl.clear(gl.COLOR_BUFFER_BIT | gl.DEPTH_BUFFER_BIT);
17
18  // 4. Request the browser to schedule the next repaint loop
19  requestAnimationFrame(render);
20 }
```

This recursive structure serves as the heartbeat of the game engine, seamlessly decoupling computational workloads from the hardware refresh speed to provide an identical arcade feel across different system setups.

2. External Libraries & Assets

2.1 Third-Party Dependencies

To fulfill the strict requirements of the project, we developed almost all the mathematics, game physics, and animation logic from scratch. However, to avoid rewriting low-level linear algebra routines and array utilities, we integrated one specific external library: **gl-matrix.js**.

2.1.1 gl-matrix.js

WebGL does not have built-in classes or functions to handle matrices or 3D vectors. Since our game requires shifting objects, rotating body parts, and projecting a 3D view onto a 2D screen, we used **gl-matrix.js** to handle these operations efficiently. From this library, our code relies exclusively on three main structures:

- **mat4**: Used in `objects.js` and `main.js` to create and multiply 4×4 transformation matrices for every object in the scene.
- **mat3**: Used in `main.js` (`renderObject` and `renderTexturedObject`) to compute normal matrices, which ensure that our surface lighting stays accurate when objects scale or rotate.
- **vec3**: Used across the entire project to define 3D coordinates for positions, velocities, camera directions, and colors.

2.2 Asset Strategy: 100% Procedural Generation

A key decision in our project was how to handle 3D models and graphics. Instead of downloading external `.obj` files or using pre-made image textures, we decided to generate everything dynamically through code during the initialization phase.

2.2.1 Geometry Generation

All the shapes in the game are calculated mathematically inside `geometry.js`. We created functions that compute vertex positions, normals, and coordinates step by step:

- **createBoxMesh**: Generates standard cubes used for the stadium borders, goals, limbs, and the player's torso.
- **createSphereMesh**: Computes the coordinates for a round shape using rings and segments. This single function allows us to render both the soccer ball and the giant heads of the players.
- **createCylinderMesh**: Calculates a tubular shape, which we use to represent the interactive power-ups floating on the pitch.

2.2.2 Texture Creation and Virtual Canvas Routing

To avoid loading external `.png` or `.jpg` files, we used the HTML5 Canvas 2D API inside `main.js` to draw our images programmatically at startup:

- **createFieldTexture:** Draws a green rectangle with alternating light and dark stripes, along with white stadium lines and a center circle.
- **createBallTexture:** Paints a white background and loops through coordinates to draw black pentagons, creating a traditional soccer ball pattern.

Once drawn on a hidden canvas, these images are uploaded directly into WebGL textures using standard filters (`gl.LINEAR_MIPMAP_LINEAR`) to keep them smooth. The following JavaScript implementation from `main.js` demonstrates exactly how the football field's visual markings and turf patterns are programmatically rendered onto an internal canvas before being bound as an active WebGL texture:

```
1 function createFieldTexture() {
2   const canvas = document.createElement('canvas');
3   canvas.width = 2048;
4   canvas.height = 1024;
5   const ctx = canvas.getContext('2d');
6
7   // Draw alternating grass stripes for the field pitch
8   for (let i = 0; i < 12; i++) {
9     ctx.fillStyle = i % 2 === 0 ? '#2b9043' : '#1f7132';
10    ctx.fillRect(i * canvas.width / 12, 0, canvas.width / 12, canvas.
        height);
11  }
12
13  // Overlay the layout chalk lines and center penalty circle
14  ctx.strokeStyle = 'rgba(255,255,255,0.92)';
15  ctx.lineWidth = 14;
16  ctx.strokeRect(18, 18, canvas.width - 36, canvas.height - 36);
17
18  ctx.beginPath();
19  ctx.arc(canvas.width / 2, canvas.height / 2, 130, 0, Math.PI * 2);
20  ctx.stroke();
21
22  return createTextureFromCanvas(canvas);
23 }
```

This approach keeps our project compact, fast, and completely self-contained within our scripts, removing any asynchronous asset-loading bottlenecks while proving that clean retro aesthetics can be engineered directly through runtime mathematics.

3. Geometric Modeling & Primitive Reuse

3.1 Abstraction of Base Shapes via Code

Instead of importing models from external design software, all the objects in our game are built programmatically inside `geometry.js`. We achieved this by writing mathematical functions that calculate the vertex positions, their alignment (normals), and texture maps on the fly using basic geometry concepts.

3.1.1 Boxes and Planes

For flat objects like the stadium floor (`createFieldMesh`) and the goalposts (`createBoxMesh`), we defined hardcoded arrays of points in 3D space. Each face of a cube is made of two triangles, so we specified the four corners of each face and used an index array (`indices`) to tell WebGL how to connect them. This keeps the geometry perfectly clean and efficient for flat surfaces.

3.1.2 Spheres and Curved Shapes via Trigonometric Latitudes

To create rounded objects like the soccer ball and the players' heads, we implemented the `createSphereMesh` function. Since computers cannot draw perfect curves, we simulated a sphere using a grid of rings and segments.

Inside a nested loop, we used basic trigonometric functions (`Math.sin` and `Math.cos`) to calculate the exact 3D coordinates (x, y, z) around the surface of the sphere based on step angles. The density of this grid is balanced so that the ball and heads look perfectly smooth during gameplay without slowing down the rendering process. The following implementation from `geometry.js` shows how these spatial coordinates and normal lighting orientation values are procedural generated at startup:

```
1 function createSphereMesh(segments = 32, rings = 16) {
2   const vertices = [];
3   const normals = [];
4   const radius = 0.5;
5
6   for (let r = 0; r <= rings; r++) {
7     const theta = (r / rings) * Math.PI;
8     const sinTheta = Math.sin(theta);
9     const cosTheta = Math.cos(theta);
10
11     for (let s = 0; s <= segments; s++) {
12       const phi = (s / segments) * Math.PI * 2;
13       const x = Math.cos(phi) * sinTheta;
14       const y = Math.sin(phi) * sinTheta;
15       const z = cosTheta;
16
17       // Store positions adjusted by unit radius and unit normal
18       // vectors
```

```
18         vertices.push(radius * x, radius * y, radius * z);
19         normals.push(x, y, z);
20     }
21 }
22 // Code continues with index linking to map triangle primitives...
23 }
```

3.2 The Reuse Approach: Instancing via Transformations

One of the most efficient choices in our architecture is how we handle memory for these shapes. WebGL can become slow if we upload a new set of vertices for every single object on the screen. To fix this, our system generates only one "unit" version of each basic shape during startup.

3.2.1 Single Topology in Memory

When the game starts, functions like `createSphereMesh` or `createBoxMesh` run exactly once. They upload a single sphere of radius 1 and a single cube of size 1 into the GPU's memory. We do not have a separate mesh for Player 1's head, another for Player 2's head, and another for the ball. They all share the exact same base sphere data.

3.2.2 Deformation via Matrices

To represent different objects using the same shared geometry, we use the model matrix of each `SceneObject` in `objects.js`. Before drawing an object, the code applies individual scaling factors to stretch or shrink the base shape:

- **The Ball:** Uses the sphere mesh scaled uniformly by its physical radius (`this.radius`).
- **The Heads:** Use the exact same sphere mesh, but scaled with different dimensions to look larger than the body.
- **Limbs and Torso:** Use the same base box mesh, scaled into elongated rectangular blocks to look like feet, legs, or bodies.

This approach means that rendering a new obstacle or modifying a player's size (like during a power-up) does not require creating new geometry; it only requires changing a few numbers in the scale matrix before drawing.

4. Affine Transformations & Hierarchical Scene Graph

4.1 The Model Matrix: Encapsulation in SceneObject

To place any 3D entity into the world, our project implements a base class called `SceneObject` within `objects.js`. This class acts as the blueprint for everything that exists in the game, including players, goals, and the ball. Instead of scattering numbers across the code, `SceneObject` groups together three essential properties for every item: its position vector, its rotation angles along the three axes, and its scale factors.

The connection to the shaders happens through the `createModelMatrix` function. Inside this method, the code uses `gl-matrix` to build a clean 4×4 transformation matrix. It starts with a blank identity matrix and applies the transformations in a strict, standard order: first translating to the world position, then rotating around the X, Y, and Z axes, and finally scaling the unit geometry. This resulting model matrix is what we send directly to the WebGL vertex shader as a uniform variable. This allows the GPU to know exactly where and how to render that specific object in space.

4.2 Hierarchical Structure of the Players

One of the most complex visual challenges of the project is representing the avatars. A player is not just a single static shape ; it is a combination of a body, a large head, and moving feet. To make them move together naturally, we designed a conceptual tree structure—a local scene graph—where the torso acts as the master or “root” element.

In this setup, the head and the feet are sub-nodes that depend directly on the body. From a code perspective, when a player moves across the pitch or jumps into the air, we only update the main coordinates of the body. Because of the hierarchical link, the head and limbs follow the body automatically without needing separate manual position updates for each individual part. This mirrors how compound objects behave in professional graphics engines. Figure 4.1 shows how the Head Soccer player is assembled from basic geometry.

4.3 Movement Propagation: Matrix Cloning over Traditional Stacks

Instead of using the old-fashioned matrix stack push and pop mechanics typical of older graphics frameworks, our project uses a modern and direct approach in JavaScript by using matrix cloning via `mat4.clone()`.

To render the limbs solidary to the character, the drawing routine first calculates the global matrix of the body. When it is time to draw a child element, like a foot that is swinging to kick the ball, the code clones the body’s matrix. This creates a separate snapshot of the parent’s current position and rotation. Then, the specific local adjustments of the foot (such as its orbit position and its kicking rotation) are multiplied on top of this cloned copy.

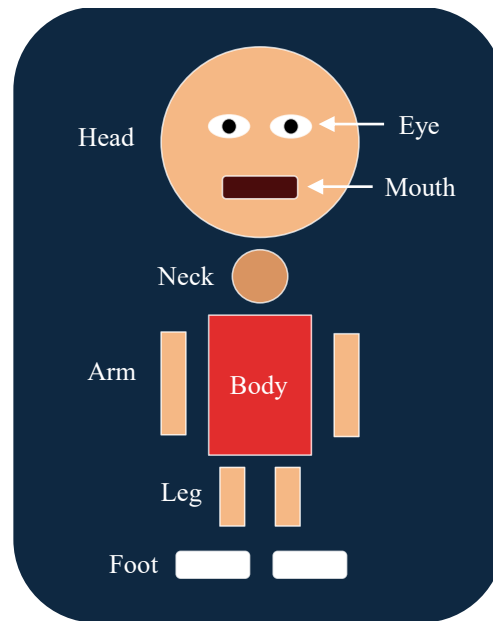


Figure 4.1: Composition of the Player from Basic Geometric Shapes

The following code section from our `main.js` file shows how this branching is implemented in our `renderLeg` function to attach and animate the upper leg and the foot seamlessly relative to the hip joint:

```

1 function renderLeg(root, hip, angle, pantsColor, shoeColor, direction) {
2   // 1. Clone the parent root matrix (Body position snapshot)
3   const legBase = mat4.clone(root);
4
5   // 2. Translate to the hip joint location relative to the torso
6   mat4.translate(legBase, legBase, hip);
7
8   // 3. Apply the dynamic angular rotation for the walking animation
9   mat4.rotateY(legBase, legBase, angle);
10
11  // 4. Position and scale the Upper Leg bone structure
12  const upperLeg = mat4.clone(legBase);
13  mat4.translate(upperLeg, upperLeg, [0, 0, -0.15]);
14  mat4.scale(upperLeg, upperLeg, [0.12, 0.12, 0.30]);
15  renderObject(upperLeg, buffers.cube, pantsColor, 0.2);
16
17  // 5. Render the Foot as a nested sub-child of the leg base matrix
18  const foot = mat4.clone(legBase);
19  mat4.translate(foot, foot, [direction * 0.07, 0, -0.36]);
20  mat4.scale(foot, foot, [0.32, 0.16, 0.11]);
21  renderObject(foot, buffers.cube, shoeColor, 0.32);
22 }

```

This cascading propagation ensures that the foot inherits the global movement and jump height of the torso perfectly, while remaining completely free to perform its own separate kicking animations without distorting the rest of the body.

5. Procedural Trigonometric Animations

5.1 Movement without an External Skeleton

In standard 3D game development, characters are usually animated using bone skeletons (rigging) designed in external software and imported as keyframe files. However, to keep our pure WebGL engine lightweight and independent of external asset loaders, we designed a completely programmatic system. All the animations in our game are calculated in real time using pure mathematics inside JavaScript.

The main advantage of this approach is that animations require zero memory storage on the GPU and can adapt dynamically to the game's state. By reading the elapsed time or the character's speed, our scripts modify the rotation and position variables of the limbs right before building the model matrices. This produces perfectly smooth movements that scale accurately with the game's frame rate.

5.2 The Walk Cycle: Sine Wave Alternation

To make the avatars look alive when moving horizontally across the pitch, we implemented a procedural walking cycle based on harmonic sine waves (`Math.sin`). When a player presses the movement keys, the game logic updates an internal animation timer that increases based on `deltaTime`.

Inside the rendering loop, this timer is passed as an angle to the sine function. Because a sine wave naturally oscillates between -1 and 1 , we use this output as a multiplier to rotate the feet back and forth. To create a realistic walking cadence where one foot moves forward while the other moves backward, we introduce a phase shift into the equation of the second foot. This simple mathematical inversion causes the limbs to alternate perfectly, simulating a natural bipedal stride without needing any pre-recorded animation clips.

5.3 The Head Bobbing Effect: Absolute Values for Vertical Impact

When a person runs, their head does not stay at a fixed height; it bounces up and down with every step. To replicate this behavior, our project introduces a phenomenon known in graphics as "Head Bobbing".

If we applied a standard sine wave to the vertical position, the head would move up on the right step and down on the left step, which looks unnatural. To fix this, our code applies the absolute value function (`Math.abs`) to the sine wave. By transforming all negative values into positive ones, the mathematical curve curves upward twice in a single full cycle. This matches human locomotion perfectly: the head reaches its peak height in the middle of each stride and drops slightly every time either the left or the right foot hits the ground.

The following snippet from our `main.js` file within the `renderPlayer` routine reveals the exact computational implementation of these synchronous, time-dependent structural oscillations:

```
1 // Inside renderPlayer(player, shirtColor, darkColor)
```

```
2  const walking = Math.sin(player.walkPhase) * 0.55;
3
4  // Compute the kicking interpolation curve based on an active action timer
5  const kick = player.kickTimer > 0
6    ? -player.direction * 1.05 * Math.sin((player.kickTimer / 0.22) * Math.
7      PI)
8    : 0;
9
10 // Apply absolute value to double the frequency of the vertical head bounce
11 const headBob = player.onGround
12   ? Math.abs(Math.sin(player.walkPhase)) * 0.025
13   : 0.045;
14
15 // Combine walking oscillations and kicking offsets for left and right legs
16 const leftAngle = walking + (player.direction < 0 ? kick : 0);
17 const rightAngle = -walking + (player.direction > 0 ? kick : 0);
```

This double-frequency bounce gives the giant heads a comical yet convincing weight and momentum during intense matches, while blending perfectly with the alternate rotation values passed onto the hierarchical matrices of the lower leg bones.

6. Advanced Lighting & Shader Pipeline

6.1 The Shader Pipeline: Data Flow and Interpolation

To calculate the visual appearance of our game, we wrote two custom shader programs in GLSL inside `main.js`: the Vertex Shader and the Fragment Shader. Understanding how data travels between them is crucial to achieving realistic graphics.

The process starts in the CPU, which sends raw vertex attributes (like positions and normal vectors) to the Vertex Shader. This shader runs once for every corner of our triangles, multiplying the coordinates by our matrices to place them into the correct 3D space. Before finishing, the Vertex Shader outputs special variables called **varying** parameters.

As WebGL turns these triangles into individual pixels on the screen (a process called rasterization), it automatically blends and interpolates these **varying** values across the entire surface. This means that our Fragment Shader, which handles the coloring per-pixel, receives smoothly changing values for every single spot on a face. This continuous data flow is what allows us to calculate pixel-perfect lighting across curved surfaces like the ball or the players' heads.

6.2 Conceptual Breakdown of the Blinn-Phong Model

Our pixel illumination relies on a custom implementation of the Blinn-Phong reflection model within our fragment shaders. This model divides light into three distinct conceptual parts:

- **Ambient Component:** A low, constant baseline light that ensures objects are never completely pitch black, even when they are not directly facing a spotlight. It simulates the scattered light inside the stadium.
- **Diffuse Component (Lambertian):** This represents light scattering evenly in all directions when it hits a matte surface. The code calculates this by evaluating the alignment between the surface normal and the light direction. The closer the surface is to facing the light source directly, the brighter it shines.
- **Specular Component:** This creates the glossy, plastic-like highlight on the ball and characters' heads. Instead of using the traditional Phong reflection vector, our Blinn-Phong shader computes the halfway vector between the light direction and the camera view direction. Checking how close the surface normal is to this halfway vector produces a much sharper and more computationally efficient reflection peak.

6.3 Multi-Light System and Distance Attenuation Code

Our stadium is illuminated by a dynamic multi-light setup processed in a loop inside the Fragment Shader. The array tracks 5 distinct light sources simultaneously: 4 fixed spotlights placed at the corners of the arena to simulate the stadium towers, and 1 dynamic light attached directly to the moving soccer ball coordinates.

Inside the shader code, we implemented a realistic attenuation loop. For every pixel being rendered, the script calculates the distance to all 5 light sources. To mimic real-world physics without overcomplicating the math, we pass this distance through a quadratic formula that softens the light's power as it travels away from its source.

The following snippet from the `fsSolid` program in `main.js` shows the exact GLSL code used inside the multi-light evaluation loop to accumulate diffusion, specular exponents, and distance falloff values:

```
1 // Loop through all 5 stadium spotlights and ball illumination variables
2 for (int i = 0; i < 5; i++) {
3     vec3 lightDir = normalize(u_lightPositions[i] - v_worldPosition);
4
5     // Compute the Euclidean distance and the physical quadratic decay
6     float distance = length(u_lightPositions[i] - v_worldPosition);
7     float attenuation = 1.0 / (1.0 + 0.08 * distance + 0.028 * distance *
8         distance);
9
10    // Lambertian diffuse component
11    float diff = max(dot(normal, lightDir), 0.0);
12
13    // Blinn-Phong reflection highlights using the halfway configuration
14    vec3 reflectDir = reflect(-lightDir, normal);
15    float spec = pow(max(dot(viewDir, reflectDir), 0.0), 52.0) *
16        u_specularStrength;
17
18    // Accumulate the final illuminated color values
19    color += (u_color * diff + vec3(spec)) * attenuation * u_lightIntensity
20    ;
21 }
```

The total lighting contribution of all 5 sources is accumulated into the final pixel color, which explains why the turf and the players visibly light up whenever the ball rolls close to them.

7. Texturing Techniques & Camera Systems

7.1 Real-Time Texture Generation via Canvas API

To keep our project independent of external image files, we designed a system that creates textures "on the fly" at startup. Inside `main.js`, we use hidden HTML5 2D canvases as drawing boards before transferring the results into WebGL memory.

For the field texture (`createFieldTexture`), the code uses canvas filling loops to paint alternating shades of green, creating a classic striped grass pattern. Then, it uses standard stroke commands to overlay crisp white chalk lines for the boundaries and the center circle. For the ball texture (`createBallTexture`), the script loops through coordinates to procedurally draw black geometric pentagons onto a white background. Once the drawings are complete, WebGL binds these canvases as raw textures, mapping them onto our 3D geometries via UV coordinates.

7.2 Camera Setup and Look-At Space

The viewpoint in our 3D stadium is managed by the `Camera` class in `view.js`, which uses the `mat4.lookAt` utility to orient our scene. This system relies on configuring three simple vectors:

- **Eye Position:** The exact 3D coordinates where the physical camera is floating in space.
- **Target Position:** The point in the world that the camera is pointing at.
- **Up Vector:** Tells the graphics card which way is up to prevent the screen from turning upside down.

By changing these three vectors based on keyboard inputs, our code lets the user swap between a flat Side View, an overhead tactical Top View, and the main dynamic Stadium View.

7.3 The Dynamic Stadium Camera Logic

The primary camera mode, called "Stadium View", implements an interpolation logic to follow the action automatically, mimicking a professional television broadcast.

Instead of remaining completely static or locking the camera rigidly to the ball—which would cause dizzying and erratic screen movements—the camera target tracks the ball smoothly. The implementation from `vuestro` script `view.js` highlights the linear mathematical adjustment applied inside the frame updates to shift the viewport seamlessly:

```
1 // Inside Camera class in view.js
2 update(gameState) {
3   // If static camera modes are chosen, bypass structural updates
4   if (!gameState || this.mode !== 'stadium') return;
5
6   // Read the current horizontal world coordinate of the soccer ball
```

```
7   const ballX = gameState.ball.position.x;
8
9   // Apply linear scaling fractions to slide the lens context softly
10  this.target.x = ballX * 0.18;
11  this.position.x = ballX * 0.08;
12 }
```

This means that as the ball flies toward a goal, the camera slides and pans smoothly along the sidelines, keeping both the players and the ball in frame without ever losing track of the center of attention.

8. Physics Engine & Collision Responses

8.1 Kinematics and Euler Integration

To power the real-time movement of the soccer ball and the players, the application implements a custom physical simulation loop based on the **Euler Integration** method. Instead of applying complex differential equations, the system computes the position of objects at every frame by accumulation.

Inside the updates, gravity acts as a continuous downward acceleration vector applied exclusively to the vertical velocity. The new speed and position of the ball are calculated step by step using the time slice (Δt):

- `velocity.z += gravity * deltaTime`
- `position.x += velocity.x * deltaTime`
- `position.z += velocity.z * deltaTime`

To simulate horizontal air drag and field resistance, a friction factor is multiplied to the velocities on every iteration. This linear approximation creates a highly predictable environment where the ball loses momentum naturally over time, providing a solid foundation for the interactive gameplay loop.

8.2 Geometric Collision Detection

Due to the intense pace of the game, calculating exact collision intersections for complex 3D polygons would be computationally expensive. To optimize performance, our engine simplifies the objects into bounding spheres and standard boxes, projecting and testing their interactions on the main 2D playing plane.

The interaction between a player and the ball relies on a circle-to-circle intersection test. The game logic continuously calculates the Euclidean distance between the center of the ball and the key hot-spots of the avatars (the torso, the large head, and the active kicking feet). If the square of the distance between these two center points becomes less than the square of their combined radii, a collision is triggered instantly. This abstract grouping guarantees a rapid evaluation rate that keeps the gameplay completely smooth and responsive.

8.3 Momentum Transfer and Kick Modifications

When a collision is validated, the physical engine overrides the trajectory of the ball to simulate an elastic bounce. The direction of the deflection is determined by calculating a unit normal vector (`nx`, `nz`) pointing from the contact point on the avatar toward the center of the ball.

The velocity transfer changes drastically depending on whether the player is simply touching the ball or performing an active kick action. If the user triggers the action key, the script flags a boolean state. The code reads this flag and applies an additional impulse multiplier along the

attack vector. The following implementation within `objects.js` shows how the game calculates this mechanical bounce and injects extra shot power during contact:

```
1 // Inside GameState collision checks in objects.js
2 const incoming = ball.velocity.x * nx + ball.velocity.z * nz;
3
4 // Reflect incoming velocity along the contact normal vector
5 if (incoming < 0) {
6     ball.velocity.x -= 1.50 * incoming * nx;
7     ball.velocity.z -= 1.50 * incoming * nz;
8 }
9
10 // Add constant push from player velocity, plus stronger impulse if kicking
11 ball.velocity.x += nx * 0.36 + extraVelocityX * 0.14;
12 ball.velocity.z += Math.max(nz * 0.58, 0.18) + extraVelocityZ * 0.08;
13
14 if (kickPower !== 0) {
15     ball.velocity.x += Math.sign(kickPower) * Math.abs(kickPower);
16     ball.velocity.z += 1.0; // Force upward lift during kicks
17 }
```

This logic ensures that defensive deflections feel heavy and mechanical, while an intentional shot results in a high-speed trajectory toward the opponent's net.

8.4 Lock Resolution: The Sandwich Effect

A typical bug in custom game physics happens when two moving solid entities trap a smaller object between them, causing the small item to pass through boundaries or crash the software because it cannot resolve the overlapping forces simultaneously. In our game, this is known as the "Sandwich Effect".

To prevent the soccer ball from getting crushed or glitched when Player 1 and Player 2 clamp it tightly at the same time, our algorithm incorporates a positional resolution rule. Right after reflecting the speed vectors, the engine calculates the deep overlap value. Instead of just changing the speed, it forcibly updates the ball's coordinates using the contact normal combined with a small separation padding.

The code snippet below shows how the engine handles this positional adjustment, pushing the ball outward to ensure it never remains stuck or clip inside an avatar's bounding box:

```
1 // Calculate distance and determine overlapping depth
2 const distance = Math.sqrt(dx * dx + dz * dz);
3 const overlap = radius - distance;
4
5 if (overlap > 0) {
6     const nx = dx / distance;
7     const nz = dz / distance;
8     const separationPadding = 0.012;
9
10    // Forcibly push the ball out of the player's volume to resolve
11    // clamping
12    ball.position.x += nx * (overlap + separationPadding);
13    ball.position.z = Math.max(ball.radius, ball.position.z + nz * (overlap
14    + separationPadding));
15 }
```

By forcing this geometric separation right before the physics frame closes, the engine guarantees that even if both avatars compress the ball on the sidelines, the sphere resolves its position cleanly by escaping upward into the open air, preserving the stability of the game world.

9. User Manual & Deployment Guide

9.1 System Requirements

The **3D Head Soccer** engine is engineered to run as a native, zero-installation web application. Because the architecture avoids external server backends or heavy middleware platforms, the entire software executes client-side within the sandbox of any standard internet browser.

The only technical prerequisite is a modern web browser that provides built-in hardware acceleration via the **HTML5 Canvas 2D** and **WebGL 1.0** specifications. Compatible platforms include recent versions of Google Chrome, Mozilla Firefox, Microsoft Edge, and Apple Safari. No third-party plugins, container runtimes, or external browser extensions are required to run the simulation at maximum performance.

9.2 Installation and Deployment Instructions

Deploying and running the project is straightforward due to its standalone design. Since all code execution is orchestrated via vanilla JavaScript modules, there is no need to compile assets or spin up local Node.js or Python hosting servers unless required by strict local storage browser security constraints.

To execute and initialize the game environment, follow these steps:

1. Extract the source directory containing the core folders: `js/`, `css/`, `libs/`, and the root file.
2. Locate the main entry point file named `index.html` at the root of the project folder.
3. Double-click or open `index.html` directly using your preferred modern web browser.
4. The browser will load the structure, instantly boot the underlying matrix algebra routines inside `libs/gl-matrix-min.js`, proceduralize the textures, and display the stadium environment within less than a second.

9.3 Interactive Game Controls Guide

The runtime loop listens continuously to asynchronous keyboard events (`keydown` and `keyup`) to update human interactions instantly without interrupting physics updates or visual rasterization. The configuration supports full two-player local matches alongside comprehensive environment configuration shortcuts.

The following comprehensive table outlines every valid real-time keyboard interaction mapped within our execution layer:

9.3.1 Environmental Modification Triggers

As shown in Table 9.1, keys outside the movement group interact directly with global system uniforms. For example, pressing the `0` key alters the light attenuation bounds in real time, shifting the scene into an ambient stadium state.

Category	Key / Input	Mechanical Action & System Response
Player 1 Controls (Left Side)	A / D	Moves the avatar horizontally across the left pitch plane.
	W	Triggers the impulse physics routine to jump into the air.
	S	Initiates the leg rotation sequence to perform an active kick.
Player 2 Controls (Right Side)	Left / Right	Moves the avatar horizontally across the right pitch plane.
	Up	Triggers the impulse physics routine to jump into the air.
	Down	Initiates the leg rotation sequence to perform an active kick.
Camera Views (Figure 9.1)	1	Swaps to the static, orthogonal Side View layout.
	2	Swaps to the top-down tactical Top View overview.
	3	Activates the dynamic Stadium View with smooth ball tracking.
Environment Settings	+ / -	Adjusts match difficulty (toggles physics variables and speed bounds).
	O (Letter)	Toggles stadium reflectors (<code>stadiumLightsOn</code>) between night/day look. (Figure 9.2)
	H	Toggles the graphical HTML overlay containing basic instructions.

Table 9.1: Complete Keyboard Mapping for Gameplay and Environment Control Configurations.

Similarly, changing views with keys 1, 2, or 3 alters the camera parameters passed onto the `mat4.lookAt` function in `view.js`. This reconfigures the user's viewpoint on the fly, showing how flexible our native WebGL architecture is during active play.

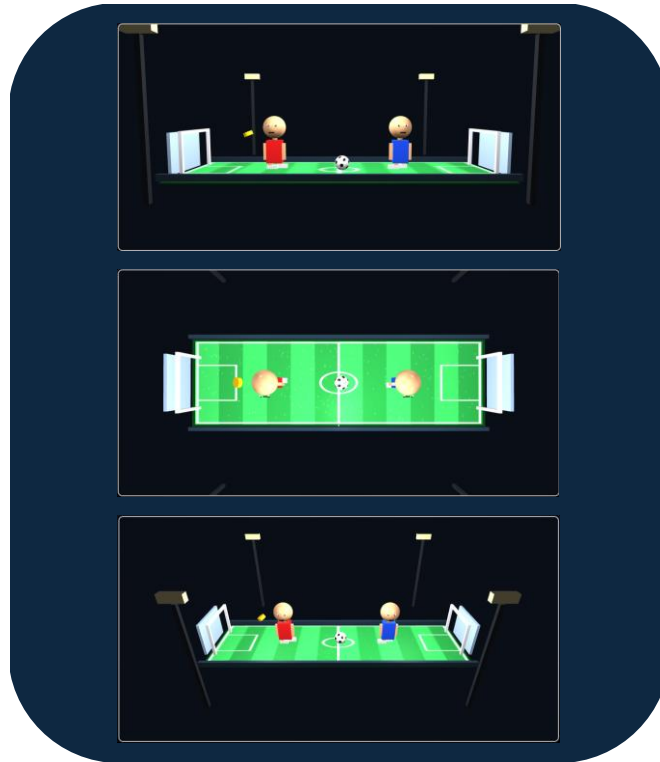


Figure 9.1: Camera views: side view above, top view in the middle, and stadium view below

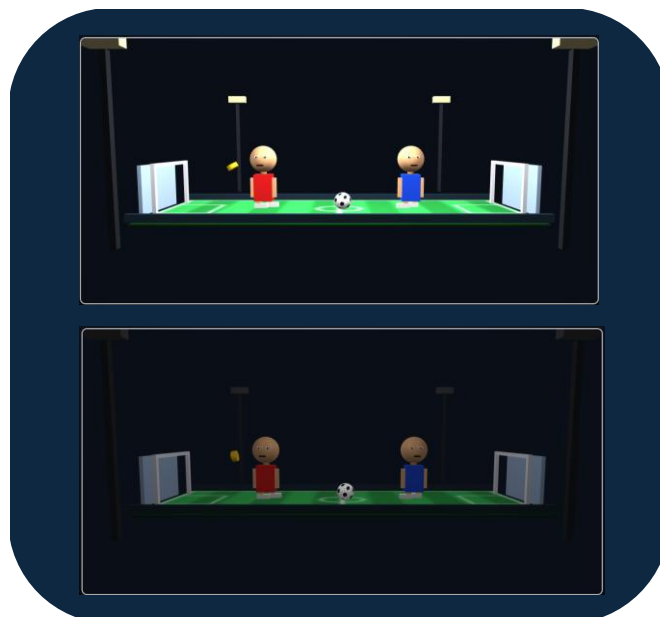


Figure 9.2: Stadium light: enabled in the upper image and disabled in the lower image